



**UNIVERSIDAD AUTÓNOMA DE
OCCIDENTE
UNIDAD REGIONAL
CULIACÁN**

INGENIERÍA DE SOFTWARE

MANUAL TÉCNICO:

**“Manual Técnico: Sistema de Gestión para Restaurantes
GastroSoft”**

NOMBRE: Héctor Paul Vizcarra Astorga

MATRÍCULA: 23040013

ASIGNATURA: Innovación de ingeniería de software

FECHA: 29/05/2026

1	Introducción técnica	3
2	Alcance, objetivos y roles	4
3	Stack tecnológico	5
4	Arquitectura general	6
5	Estructura de carpetas	7
6	Instalación y ejecución local	8
7	Configuración de Supabase	10
8	Módulos del sistema	11
9	Hooks y lógica de datos	14
10	Modelo de base de datos	16
11	Flujos internos principales	21
12	Seguridad, pruebas y despliegue	24
13	Mantenimiento, errores y glosario	27



1. Introducción técnica

GastroSoft es una aplicación web para la gestión operativa de restaurantes. El sistema centraliza comandas, mesas, pagos, inventario, proveedores, promociones, clientes, reservaciones, personal, caja, reportes y configuración.

Este manual técnico describe cómo está construido el sistema, qué tecnologías utiliza, cómo se instala, cómo se comunica con Supabase y cómo se organiza la lógica interna del proyecto.

Tipo de sistema

Aplicación web de una sola página construida con React y Vite.

Persistencia

Base de datos PostgreSQL administrada mediante Supabase y consumida con el SDK de JavaScript.

Interfaz

Dashboard modular con barra superior, navegación por módulos y componentes reutilizables.

Estado de datos

Uso de hooks personalizados para consultar, insertar, actualizar y sincronizar información.

El documento está pensado para desarrolladores, administradores técnicos y responsables de mantenimiento del sistema.



2. Alcance, objetivos y roles

Alcance funcional

El sistema cubre operación de sala, administración de productos, gestión de inventario, pagos, caja, reportes y control de personal.

Área	Funciones cubiertas
Operación	Comandas, mesas, cuentas separadas, reservaciones y pagos.
Administración	Menú, promociones, personal, proveedores y configuración.
Control	Inventario, movimientos, caja y reportes.

Roles de usuario

La aplicación filtra módulos según rol desde el frontend. Los roles detectados en la lógica principal son administrador, gerente, mesero y cocinero.

Rol	Acceso esperado
Administrador	Acceso completo a módulos de operación, administración y configuración.
Gerente	Acceso a módulos operativos y administrativos, excluyendo configuración sensible.
Mesero / Cocinero	Acceso limitado principalmente a comandas, menú y pagos.



3. Stack tecnológico

Capa	Tecnología	Uso
Frontend	React 19.2.5	Construcción de componentes, estado y vistas dinámicas.
Bundler	Vite 8.0.9	Servidor de desarrollo y compilación para producción.
Base de datos	Supabase / PostgreSQL	Persistencia de comandas, usuarios, inventario y demás tablas.
SDK	@supabase/supabase-js 2.106.0	Consultas directas desde el frontend hacia Supabase.
Estilos	CSS y estilos inline	Diseño visual del dashboard y módulos.
Lint	ESLint 9.39.4	Revisión de calidad básica del código.

Scripts disponibles

```
npm run dev      # Ejecuta servidor local de desarrollo
npm run build    # Genera versión optimizada en dist/
npm run lint     # Ejecuta análisis estático
npm run preview  # Previsualiza build de producción
```

El proyecto no utiliza backend Node propio para la lógica de negocio; las operaciones de datos se realizan mediante Supabase desde hooks de React.



4. Arquitectura general



Componentes base

Archivo	Responsabilidad
App.jsx	Controla autenticación, usuario activo, rol y módulo activo.
Header.jsx	Barra superior, navegación, perfil y cierre de sesión.
Dashboard.jsx	Renderiza el módulo correspondiente según activeModule.
useSupabase.js	Centraliza consultas y operaciones contra Supabase.



GastroSoft

5. Estructura de carpetas

```
Sistema-Comandas/  
├─ public/  
│   ├── LogoGastroSoftHeader.png  
│   └─ MenuImagenes/  
├─ src/  
│   ├── components/  
│   │   ├── Header.jsx  
│   │   ├── Dashboard.jsx  
│   │   └─ modules/  
│   ├── data/  
│   │   └─ modulesData.js  
│   ├── hooks/  
│   │   └─ useSupabase.js  
│   ├── styles/  
│   ├── supabase.js  
│   ├── App.jsx  
│   └─ main.jsx  
├─ docs/  
├─ package.json  
└─ vite.config.js
```

Carpeta	Descripción
public	Recursos estáticos: logo e imágenes de productos.
src/components/modules	Componentes de cada módulo funcional.
src/hooks	Hooks que manejan datos y llamadas a Supabase.
src/styles	Hojas CSS por módulo y estilos compartidos.
docs	Documentación generada: manuales HTML y PDF.



6. Instalación local

Requisitos previos

- Node.js instalado.
- NPM disponible en terminal.
- Acceso al proyecto Supabase configurado.
- Repositorio o carpeta del proyecto GastroSoft.

Pasos de instalación

```
# 1. Entrar a la carpeta del proyecto
cd Sistema-Comandas

# 2. Instalar dependencias
npm install

# 3. Ejecutar en modo desarrollo
npm run dev

# 4. Abrir la URL local indicada por Vite
```

Compilación de producción

```
npm run build
```

El resultado se genera en la carpeta **dist/**, lista para publicarse en un servidor web compatible con aplicaciones estáticas.



6.1 Ejecución y validación

Validación básica después de instalar

Prueba	Resultado esperado
npm run dev	Vite inicia sin errores y muestra URL local.
Pantalla login	Se muestra el acceso de GastroSoft.
Inicio de sesión	Usuario válido entra al dashboard o módulo inicial.
Cambio de módulo	La barra superior permite navegar entre módulos permitidos.
npm run build	Se genera dist/ sin errores de compilación.

Comandos de mantenimiento

```
npm run lint  
npm run build  
npm run preview
```

Si la aplicación carga pero no muestra datos, revisar conexión a Supabase, permisos de tablas y usuario autenticado.



7. Configuración de Supabase

La conexión con Supabase se define en **src/supabase.js**. Este archivo crea un cliente con **createClient** del SDK oficial.

```
import { createClient } from '@supabase/supabase-js'

const SUPABASE_URL = '...'
const SUPABASE_KEY = '...'

export const supabase = createClient(SUPABASE_URL, SUPABASE_KEY)
```

Recomendación técnica

Para producción, se recomienda mover URL y key a variables de entorno de Vite para evitar dejarlas directamente en el código fuente.

```
VITE_SUPABASE_URL=...
VITE_SUPABASE_ANON_KEY=...
```

Tablas consumidas por el frontend

El sistema consulta tablas como comandas, detalles_comanda, mesas, usuarios, asistencia, inventario, proveedores, promociones, reservaciones y cortes_caja.



8. Módulos del sistema

Los módulos disponibles se definen en **src/data/modulesData.js**. Cada módulo contiene un identificador, nombre e icono.

ID técnico	Módulo	Componente principal
dashboard	Dashboard	DashboardModule
comandas	Comandas	Comandas
menu	Menú	MenuModule
mesas	Mesas	MesasModule
personal_asistencia	Personal / Asistencia	PersonalAsistenciaModule
pagos	Pagos	Pagos
inventario	Inventario	InventarioModule



8.1 Módulos administrativos

ID técnico	Módulo	Función
proveedores	Proveedores	Alta, edición, productos y compras de proveedores.
promociones	Promociones	Gestión de descuentos, combos y ofertas.
clientes	Clientes / Reservaciones	Reservaciones, clientes, mesas reservadas y comandas ligadas.
reportes	Reportes	Consulta de ventas, comandas y datos operativos.
caja	Caja	Cortes, totales y registros del turno.
configuracion	Configuración	Ajustes administrativos del sistema.

Renderizado de módulos

Dashboard.jsx decide qué componente renderizar según el valor de **activeModule**.

```
{activeModule === 'comandas' && (  
  <Comandas ... />  
)}
```



8.2 Control de acceso por rol

App.jsx filtra los módulos disponibles según el rol del usuario activo. El usuario se conserva en localStorage para mantener sesión.

Elemento	Descripción técnica
gastrosoft_user	Clave usada en localStorage para persistir usuario actual.
currentUser	Estado React con datos del usuario autenticado.
userRole	Rol usado para filtrar módulos disponibles.
activeModule	Módulo actual renderizado por Dashboard.jsx.

```
const modulosMesero = modulesData.filter(m =>
  ['comandas', 'menu', 'pagos'].includes(m.id)
)
```

Para seguridad robusta, los permisos deben reforzarse también en Supabase mediante políticas RLS, no solo en frontend.



9. Hooks y lógica de datos

La lógica de consulta y actualización se concentra en **src/hooks/useSupabase.js**. Cada hook encapsula un área funcional.

Hook	Tablas principales	Responsabilidad
useMenu	categorias, productos	Carga categorías y productos del menú.
useComandas	comandas, detalles_comanda, mesas	CRUD de comandas y detalles.
useMesas	mesas	Alta, edición y estado de mesas.
usePersonal	usuarios, asistencia	Empleados, roles y asistencia.
useClientes	reservaciones, mesas, comandas	Reservaciones y comanda asociada.
useInventario	inventario, movimientos_inventario	Stock y movimientos.



9.1 Hooks administrativos

Hook	Tablas principales	Responsabilidad
useProveedores	proveedores, productos_proveedor, compras_proveedor, inventario	Proveedores, compras y actualización de stock.
usePromociones	promociones	Promociones, estados y actualización.
useCaja	cortes_caja	Historial y cierre de caja.
useReportes	comandas, detalles_comanda	Datos agregados para reportes.

Patrón general de hooks

```
const [datos, setDatos] = useState([])

useEffect(() => {
  fetchDatos()
}, [])

async function fetchDatos() {
  const { data } = await supabase.from('tabla').select('*')
  setDatos(data || [])
}
```



10. Modelo de base de datos I

Tabla	Campos relevantes	Uso
usuarios	id, nombre, usuario, contrasena, rol, estado, foto_perfil	Login, empleados, roles y perfil.
mesas	id, numero, capacidad, ubicacion, estado	Control de mesas disponibles, ocupadas o reservadas.
categorias	id, nombre	Agrupación de productos del menú.
productos	id, nombre, precio, categoria, imagen, estado	Catálogo visible en menú y comandas.
asistencia	id, id_usuario, entrada, salida, fecha	Registro de asistencia del personal.

Relaciones principales

- Un usuario puede aparecer como mesero o empleado responsable.
- Una mesa puede estar ligada a comandas y reservaciones.
- Los productos pertenecen a categorías del menú.



10.1 Modelo de base de datos II

Tabla	Campos relevantes	Uso
comandas	id, numero_comanda, id_mesa, id_mesero, estado, subtotal, descuento, impuesto, total, id_reservacion, cuenta_separada	Orden principal del cliente.
detalles_comanda	id, id_comanda, nombre_item, cantidad, precio_unitario, subtotal, observaciones	Productos incluidos en una comanda.
reservaciones	id, cliente, fecha, hora, personas, id_mesa, telefono, estado	Reservas y relación con mesas.
promociones	id, nombre, descripcion, tipo, valor, estado	Descuentos y combos.

Relación comanda-reservación

Al crear una reservación, el sistema puede crear una comanda asociada con **id_reservacion**, **id_mesa** y límite de cuentas según número de personas.



10.2 Modelo de base de datos III

Tabla	Campos relevantes	Uso
inventario	id, nombre, cantidad, unidad, cantidad_minima, cantidad_maxima, id_proveedor, estado	Control de ingredientes y existencias.
movimientos_inventario	id, id_ingredient, tipo, cantidad, empleado, fecha	Entradas y salidas de stock.
proveedores	id, nombre, telefono, correo, direccion, estado	Datos de proveedores.
productos_proveedor	id, id_proveedor, nombre, precio, unidad, estado	Productos ofrecidos por proveedor.
compras_proveedor	id, id_proveedor, total, fecha, estado	Compras registradas.
cortes_caja	id, fecha, total_ventas, efectivo, tarjeta, usuario	Cierres y cortes de caja.



10.3 Diagrama relacional lógico



Reglas de integridad recomendadas

- No eliminar físicamente datos críticos de ventas; preferir estados como cancelado o inactivo.
- Validar que una comanda pagada no pueda modificarse sin autorización.
- Actualizar estado de mesa al crear, pagar o cancelar comanda.
- Registrar todo movimiento de inventario con empleado responsable.



10.4 Diccionario de estados

Entidad	Estados usados	Significado
Mesa	disponible, ocupada, reservada	Disponibilidad operativa de mesa.
Comanda	pendiente, Pagado, Cancelado	Estado de orden y pago.
Inventario	normal, bajo, agotado	Nivel de stock respecto al mínimo.
Proveedor	activo, inactivo	Proveedor disponible o deshabilitado.
Promoción	activa, inactiva	Promoción disponible o no aplicable.
Usuario	activo, inactivo	Permite o bloquea acceso al sistema.

Conviene normalizar mayúsculas/minúsculas de estados para evitar comparaciones inconsistentes en frontend.



11. Flujo: inicio de sesión



Consulta base

```
supabase
  .from('usuarios')
  .select('*')
  .eq('usuario', username)
  .eq('contrasena', password)
  .eq('estado', 'activo')
  .single()
```



11.1 Flujo: creación de comanda



Tablas afectadas

- comandas
- detalles_comanda
- mesas



11.2 Flujos: inventario y reservaciones

Inventario



Reservación





12. Eventos y sincronización

El sistema utiliza eventos del navegador para notificar cambios entre módulos y refrescar datos.

Evento	Uso
comandas:changed	Indica que cambió una comanda o sus detalles.
mesas:changed	Indica cambio de estado o datos de una mesa.
reservaciones:changed	Indica creación o modificación de reservaciones.
inventario:changed	Indica entrada, salida o actualización de inventario.

```
window.dispatchEvent(new Event('comandas:changed'))  
window.addEventListener('reservaciones:changed', recargarReservaciones)
```

Este enfoque es práctico para una aplicación pequeña/mediana; si crece, puede evaluarse un estado global formal o suscripciones realtime de Supabase.



13. Seguridad técnica

Aspecto	Estado actual	Recomendación
Autenticación	Consulta directa a tabla usuarios.	Usar Supabase Auth o hashing de contraseñas.
Credenciales	Cliente Supabase configurado en frontend.	Mover valores a variables de entorno y no exponer claves sensibles.
Autorización	Filtro de módulos en frontend.	Aplicar políticas RLS en Supabase.
Datos críticos	Operaciones CRUD directas.	Validar reglas en base de datos o funciones seguras.

Buenas prácticas

- No guardar contraseñas en texto plano.
- No eliminar registros críticos sin respaldo.
- Limitar permisos de la anon key usando RLS.
- Registrar auditoría en operaciones sensibles como pagos, caja e inventario.



14. Pruebas y validación

Pruebas manuales recomendadas

Módulo	Prueba	Resultado esperado
Login	Usuario válido e inválido.	Permite o bloquea acceso correctamente.
Comandas	Crear, editar, eliminar y pagar.	Actualiza mesa y totales.
Reservaciones	Crear, editar y cancelar.	Reserva/libera mesa y sincroniza comanda.
Inventario	Entrada y salida.	Actualiza cantidad y movimiento.
Caja	Corte de turno.	Registra corte en cortes_caja.

Validación automática mínima

```
npm run lint  
npm run build
```



15. Despliegue y producción

Build

```
npm run build
```

El comando genera una versión estática en **dist/**. Esa carpeta puede publicarse en servicios como Netlify, Vercel, hosting estático o un servidor propio configurado para SPA.

Checklist antes de publicar

- Verificar variables de entorno.
- Ejecutar build sin errores.
- Confirmar conexión con Supabase.
- Probar login y navegación por roles.
- Probar creación de comanda y pago en entorno controlado.
- Revisar políticas de seguridad en Supabase.

Si se publica como SPA, el servidor debe redirigir rutas desconocidas a index.html.



16. Mantenimiento y respaldos

Mantenimiento periódico

- Revisar dependencias con frecuencia.
- Respalidar base de datos de Supabase.
- Revisar logs o errores reportados por usuarios.
- Validar que cortes de caja y pagos cuadren.
- Depurar productos, proveedores o usuarios inactivos.

Respaldo recomendado

Elemento	Frecuencia sugerida
Base de datos Supabase	Diaria o semanal, según operación.
Repositorio del proyecto	Cada cambio importante.
Imágenes del menú	Cuando se agreguen o cambien productos.
Manuales y documentación	Después de cambios funcionales.



17. Solución de problemas

Problema	Causa probable	Solución
No inicia sesión	Credenciales incorrectas o usuario inactivo.	Validar tabla usuarios y estado activo.
No carga información	Fallo de conexión Supabase o permisos.	Revisar consola, URL, key y RLS.
No actualiza mesa	Evento no disparado o update fallido.	Validar mesas:changed y respuesta de Supabase.
Inventario incorrecto	Movimiento duplicado o salida mal capturada.	Revisar movimientos_inventario y cantidad actual.
Build falla	Error de JSX o import faltante.	Ejecutar npm run build y corregir línea indicada.

Diagnóstico rápido

```
npm run build
npm run lint
# Revisar consola del navegador
# Revisar errores devueltos por Supabase
```



18. Glosario técnico

Término	Definición
SPA	Aplicación de una sola página, renderizada desde frontend.
Hook	Función de React que maneja estado, efectos o lógica reutilizable.
Supabase	Plataforma backend basada en PostgreSQL y APIs automáticas.
RLS	Row Level Security; políticas de acceso por fila en PostgreSQL/Supabase.
CRUD	Crear, leer, actualizar y eliminar registros.
Build	Versión optimizada del sistema para producción.
Evento	Notificación interna usada para recargar módulos relacionados.

Conclusión

GastroSoft está construido como una aplicación web modular con React, Vite y Supabase. Su mantenimiento depende de conservar ordenada la estructura de componentes, reforzar seguridad en Supabase y validar los flujos críticos: login, comandas, pagos, inventario, reservaciones y caja.